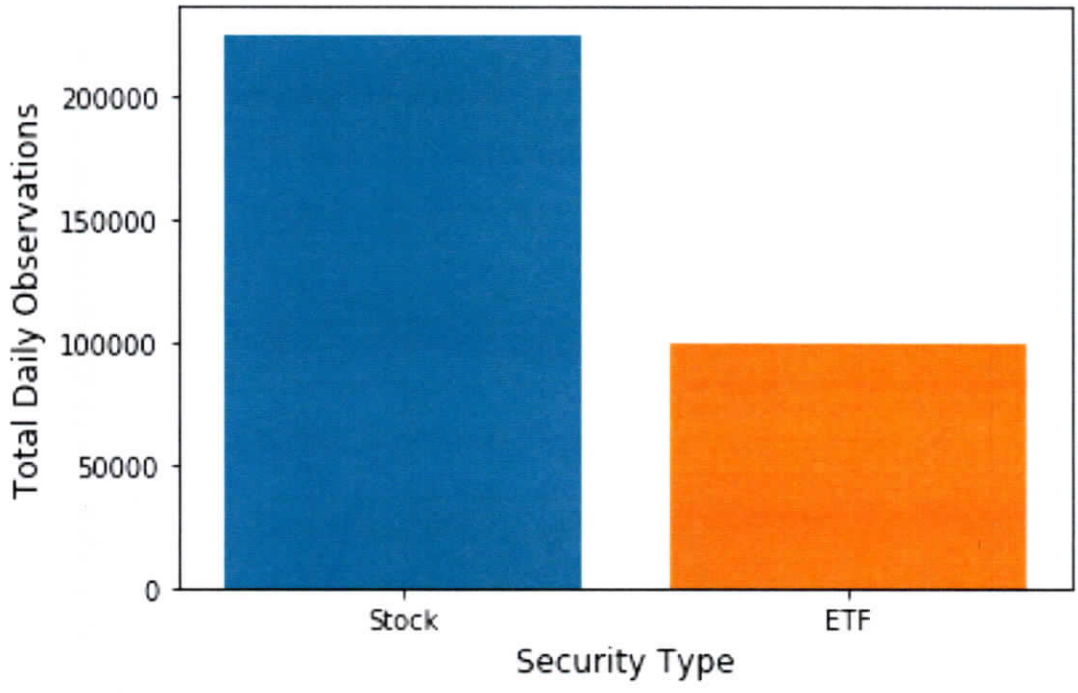
Distributions of Categorical Variables

Basic bar charts can be created to inspect the distribution of a categorical variable, using `countplot()`:

```
In [9]: plt.figure(figsize=[6,4])
        ax = sns.countplot(x="Security", data=fulldata)
        plt.xlabel("Security Type", size=12)
        plt.ylabel("Total Daily Observations", size=12)
        plt.show()
```



For the next plots, we will restrict to data that do not have missing values on `McapRank`, and are of type `Stock`:

```
In [10]: fulldataStock = fulldata[\
             (fulldata["Security"] == "Stock") \
               & (fulldata["McapRank"].notnull())]
```

Exercise: Consider the command below. What is the result?

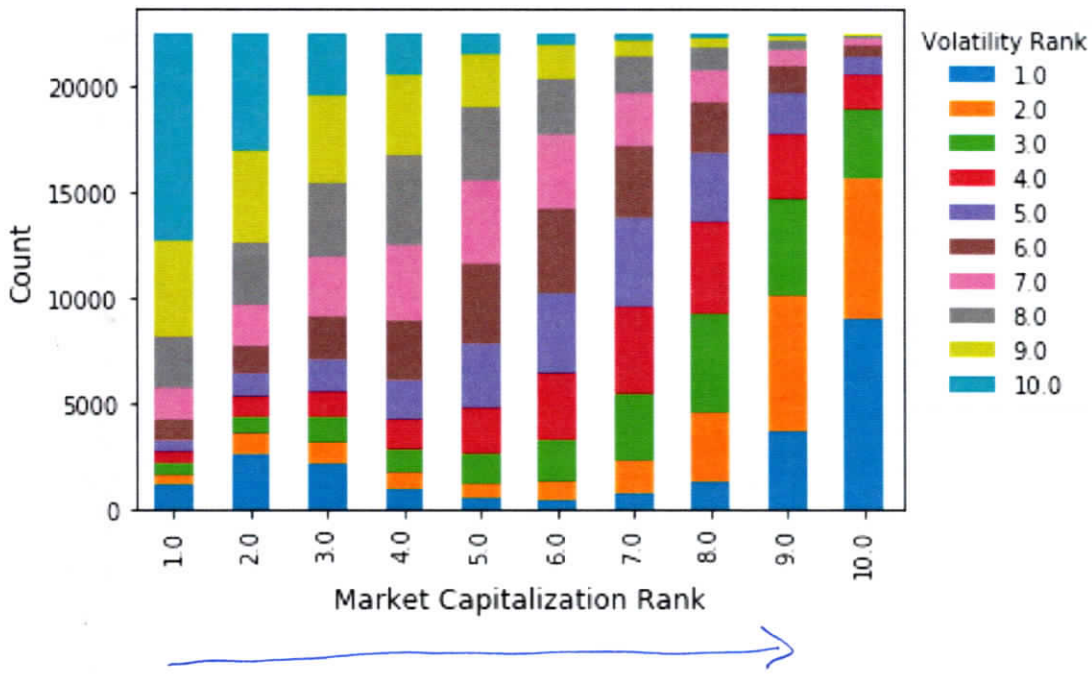
```
In [11]: df = fulldataStock.groupby(['McapRank',\
             'VolatilityRank'])['McapRank'].\
             count().unstack()
```

Two-way table comparing market cap rank and volatility rank

The **stacked bar chart** created below visualizes these data.

```
In [26]: plt.figure(figsize=[9,7])
         df.plot(kind='bar', stacked=True, legend=False)
         plt.xlabel("Market Capitalization Rank",
             size=12)
         plt.ylabel("Count", size=12)
         plt.legend(title="Volatility Rank",
             bbox_to_anchor=(1.0, 1.0))
         plt.show()
```

```
<Figure size 648x504 with 0 Axes>
```

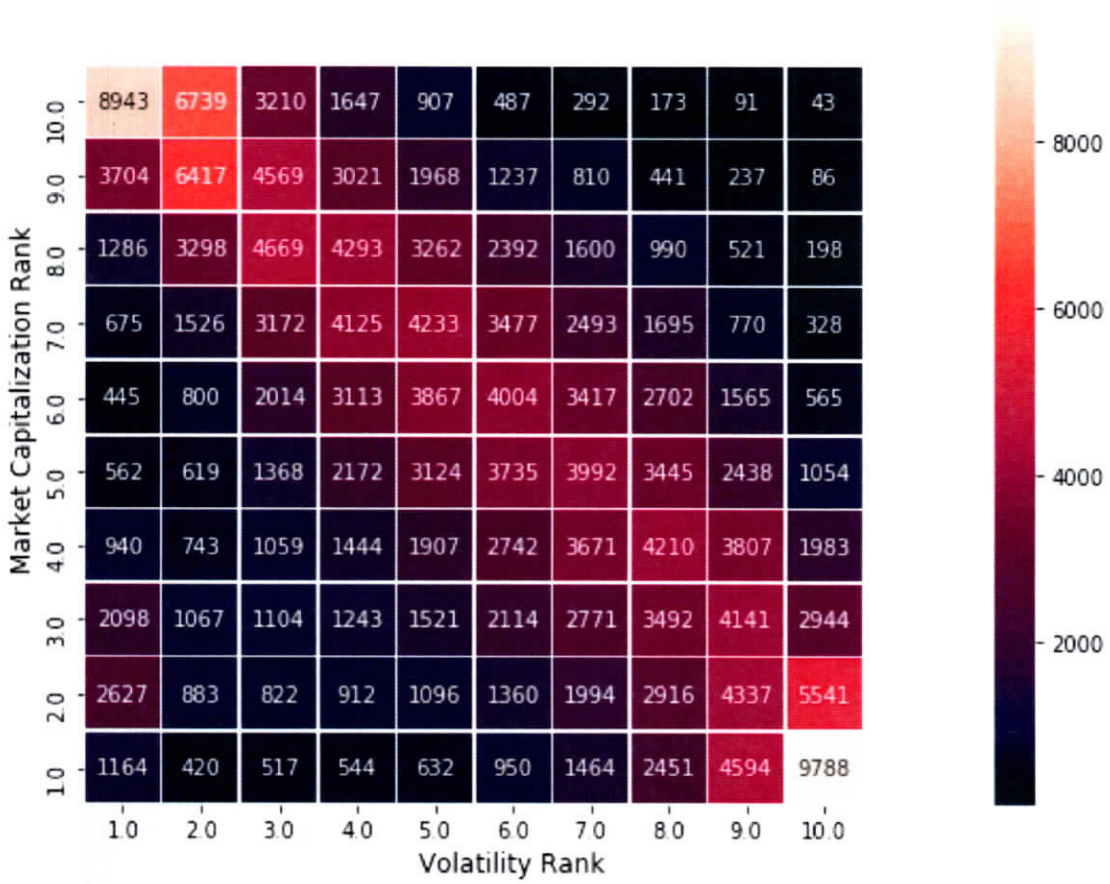


Exercise: In this case, the two variables are ordinal. Would the plot make sense if either variable was categorical (and not ordinal)?

It would still make sense, but if one variable is ordinal and one is categorical, makes more sense to put ordinal variable as the "Y" variable, because the "stacking" implies an ordering

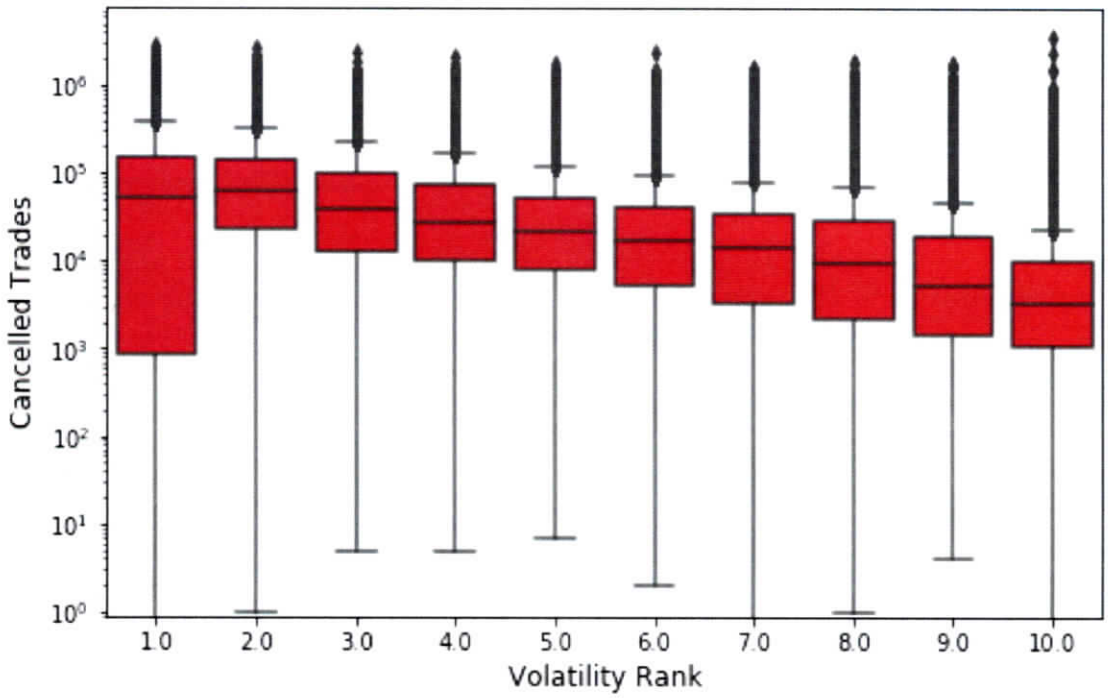
The **heat map** created below gives another visualization of the same data.

```
In [23]: plt.figure(figsize=[9,7])
         ax = sns.heatmap(df, annot=True,
            fmt='d', linewidths=.5)
         ax.invert_yaxis()
         plt.xlabel("Volatility Rank", size=12)
         plt.ylabel("Market Capitalization Rank", size=12)
         ax.set(xlim=(0, 11), ylim=(0, 11))
         plt.show()
```



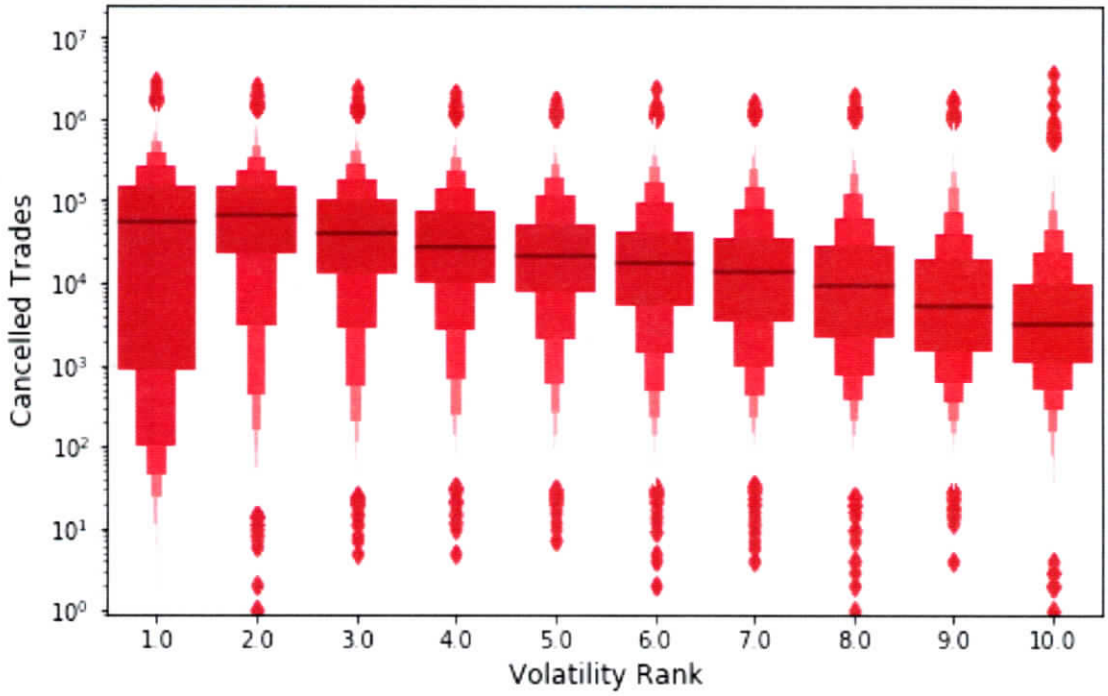
When comparing a ratio variable versus a categorical or ordinal variable, **side-by-side boxplots** are a standard choice.

```
In [15]: plt.figure(figsize=[8,5])
         ax = sns.boxplot(x="VolatilityRank", y="Cancels",
             data=fulldataStock,color="red")
         ax.set(yscale="log", ylim=(0.9, None))
         plt.xlabel("Volatility Rank",size=12)
         plt.ylabel("Cancelled Trades",size=12)
         plt.show()
```



Seaborn includes **boxenplots** to provide a similar, but more sophisticated, look at the data.

```
In [16]: plt.figure(figsize=[8,5])
         ax = sns.boxenplot(x="VolatilityRank",
              y="Cancels",
              data=fulldataStock,color="red")
         ax.set(yscale="log", ylim=(0.9, None))
         plt.xlabel("Volatility Rank",size=12)
         plt.ylabel("Cancelled Trades",size=12)
         plt.show()
```



Facets

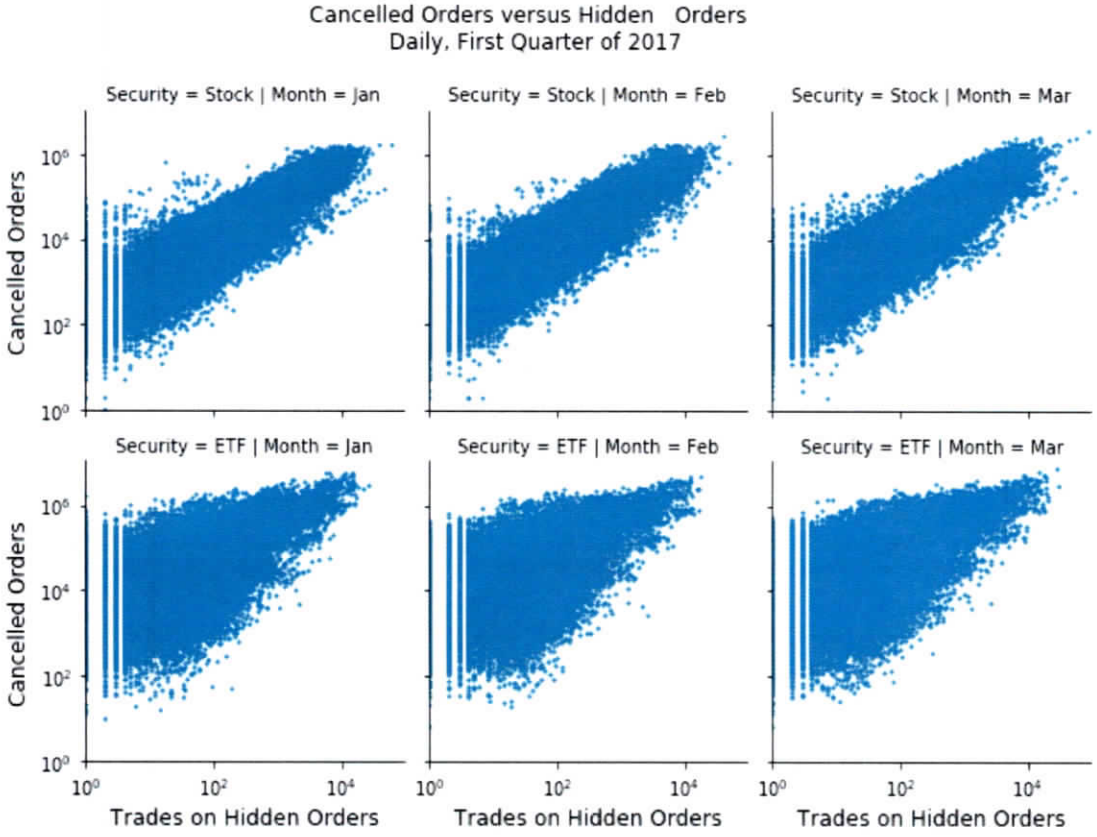
Facets refer to the plots that form a grid. Typically one or more categorical/ordinal variables vary on the rows and columns of the grid.

The example below compares the relationship between the number of cancelled orders and trades on hidden orders, as both a function of security type and month.

Note first how the `Month` column is added to the data frame.

```
In [17]: fulldata["Month"] = fulldata["Date"].\
             map(lambda x: x.strftime("%b"))
         plt.figure(figsize=[8,5])
         g = sns.FacetGrid(fulldata, col="Month",
             row="Security")
         g.map(plt.scatter, "Hidden", "Cancels", s=2)
         g.set(xscale="log", yscale="log",
             xlim=(1, None), ylim=(1, None))
         g.fig.suptitle("Cancelled Orders versus Hidden\
             Orders \n Daily, First Quarter of 2017", y=1.08)
         g.set_xlabels("Trades on Hidden Orders",
             size=12)
         g.set_ylabels("Cancelled Orders",
             size=12)
         g.add_legend()
         plt.show()
```

```
<Figure size 576x360 with 0 Axes>
```



Exercise: Explore https://seaborn.pydata.org/examples/index.html to discover additional ideas for visualization. We will be utilizing some of these approaches as they become relevant.

Part 3: Introduction to Modelling

A fundamental step in many data analyses is to model raw, observable data, i.e. to replace the complex processes that generate these data with a mathematically and computationally simpler fiction.

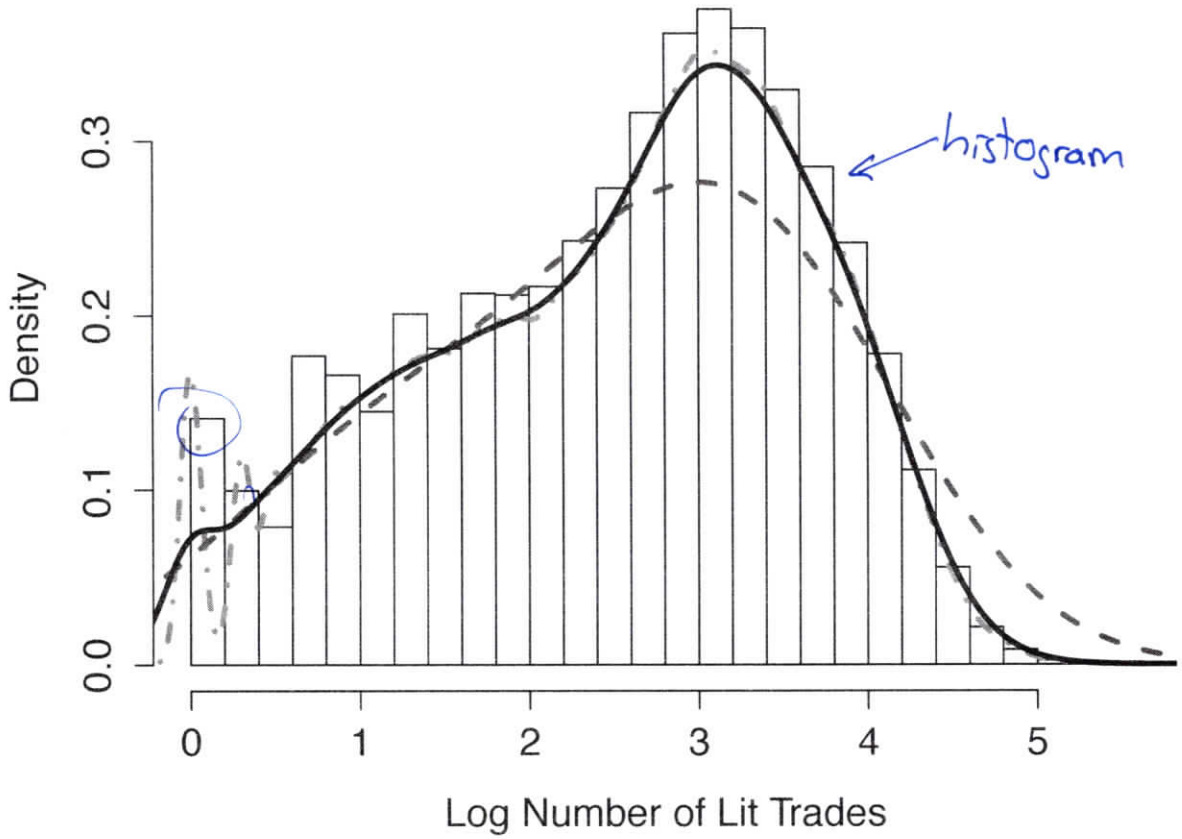
You have likely heard the expression, "All models are wrong; some are useful."[1] That is a good mentality to take towards the modelling process: We do not believe that the data were actually generated by, for example, drawing from a normal distribution. Instead, it **may** be the case that the normal provides a sufficiently accurate approximation to that process.

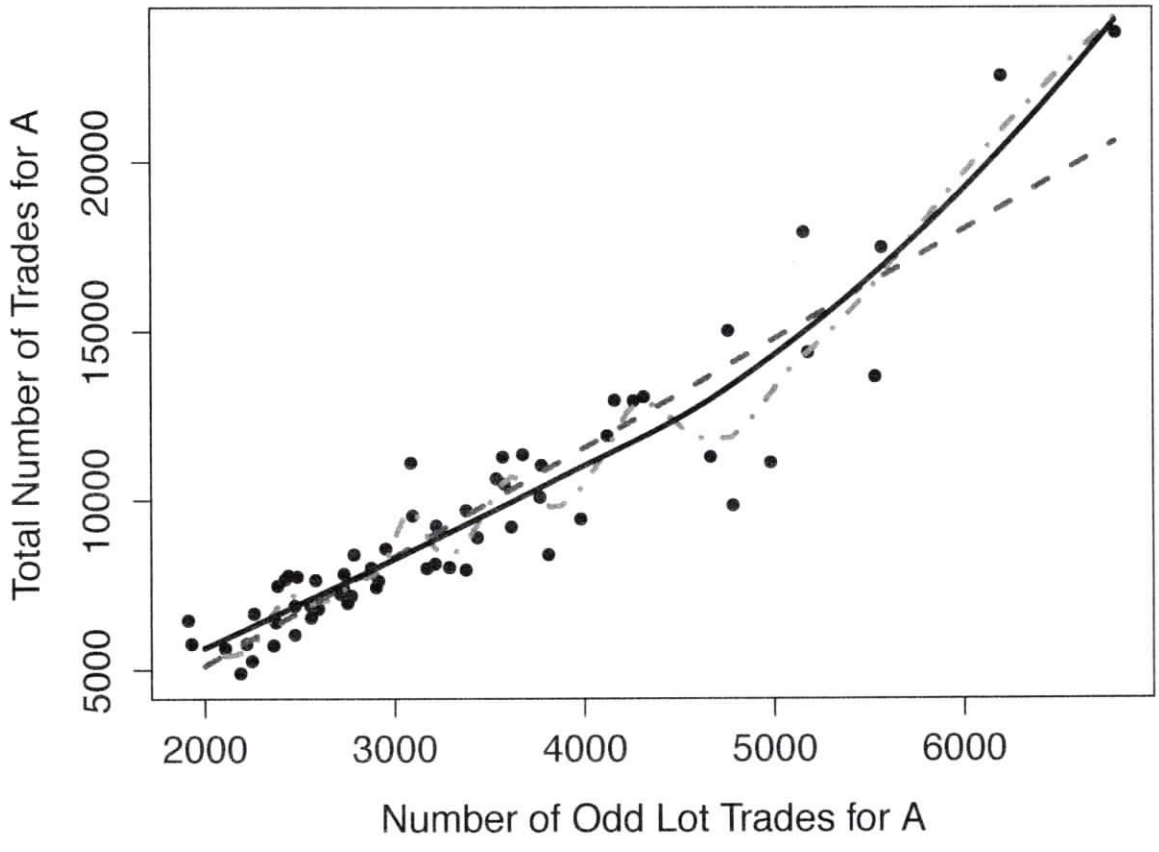
[1] For a history of this saying, see https://en.wikipedia.org/wiki/All_models_are_wrong.

A key heuristic in the statistical modelling process is that of smoothness.

We believe that most relationships, and most distributions, are fundamentally smooth, i.e., we don't try to fit to every little fluctuation that our sample may possess.

Of course, this idea can be taken too far, and oversmoothing can lead to missing important features that are present in data.


histogram
Density
0.0
0.1
0.2
0.3
0
1
2
3
4
5
Log Number of Lit Trades


Total Number of Trades for A
5000
10000
15000
20000
2000
3000
4000
5000
6000
Number of Odd Lot Trades for A

Exercise: Explain how the three proposed models on the previous slides seem to perform, in regards to the appropriate amount of smoothing of the histogram and scatter plot.

In each case, the dashed (blue) line is the smoothest model. In fact, in both cases the model is oversmoothing, masking seemingly important features in the data
The dash-dotted line (red) is the least smooth. It is fitting to fluctuations that are likely "artifacts"
The solid (black) line is a compromise between these two extremes

Exercise: Explain why overfitting is such a significant concern.